CSA 0407 - Operating System &
Short Answer Test

Name: O. Siva Charan
Reg no: 192525039

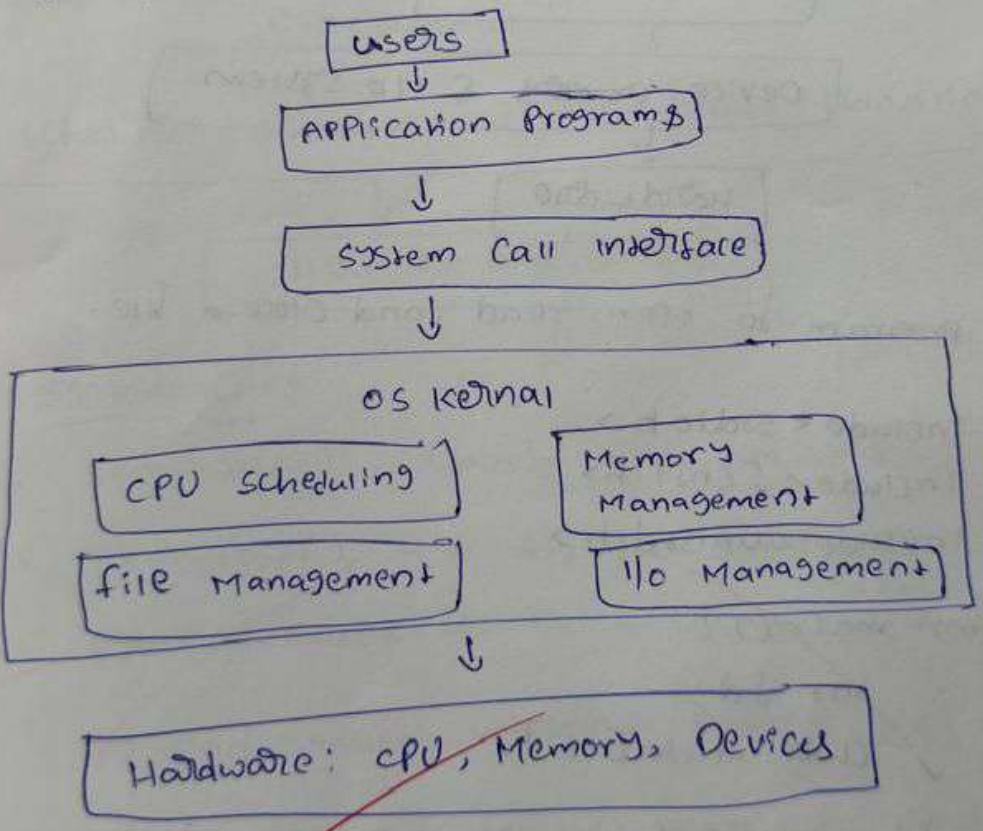
23

1) Modern operating system

(a) Types of os and justification

- Multi-user OS: Allows multiple users to access the system simultaneously.
- Multiprocessing OS: uses multiple CPUs or cores to execute processes.
- Multitasking OS: Runs multiple processes at the same time.
- Time-Sharing OS: Shares CPU time among users and processes.

(b) os structure diagram




(c) System calls

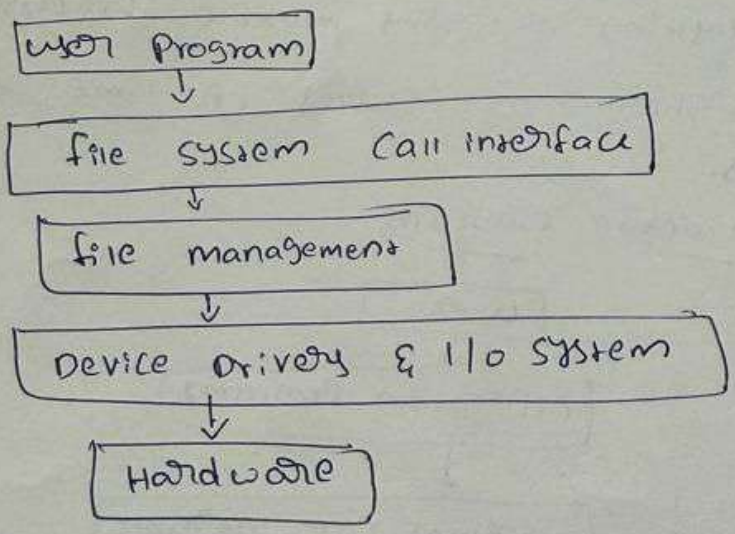
1) fork() - Creates a new Process.
2) exec() - Executes a new Program in the Process.
3) wait() - It waits for the other Programs.
4) Exit() - Terminates the Processes.

2)

(a) File & Device management :

- File management : open(), read(), write(), close(), unlink()
- Device management : ioctl(), read(), write(), close(), open()
- Information management : stat(), fstat()
- Protection : chmod(), access()

(b)




(c) C program to open, read, and close a file.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char data[100];

    fd = open("file.txt", O_RDONLY);
    if (fd == -1) return 1;

    read(fd, data, sizeof(data));
    close(fd);
    return 0;
}
```

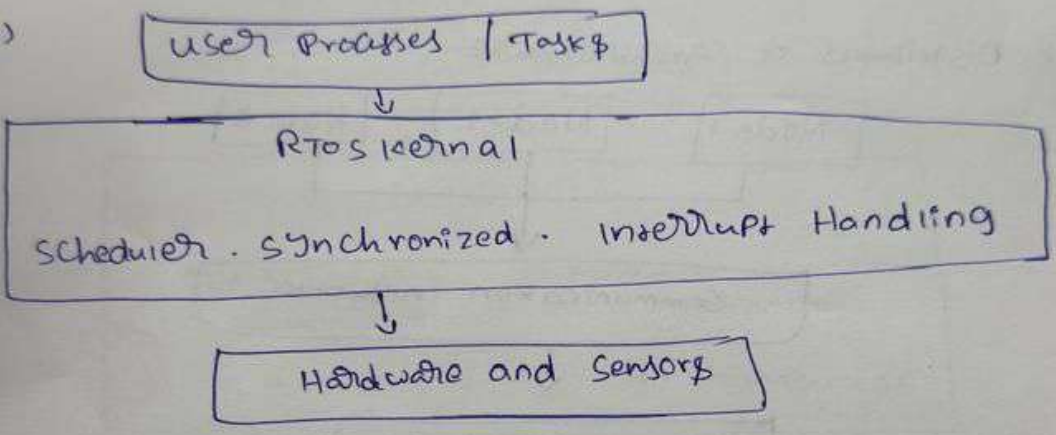
9 Real-Time OS

(a) Type of OS and suitability

A Real-Time OS (RTOS) is suitable for controlling industrial machinery.

- Gives quick and predictable responses.
- Meets strict timing deadlines.
- Supports priority based scheduling.
- Provides synchronization between Processes.

(b)



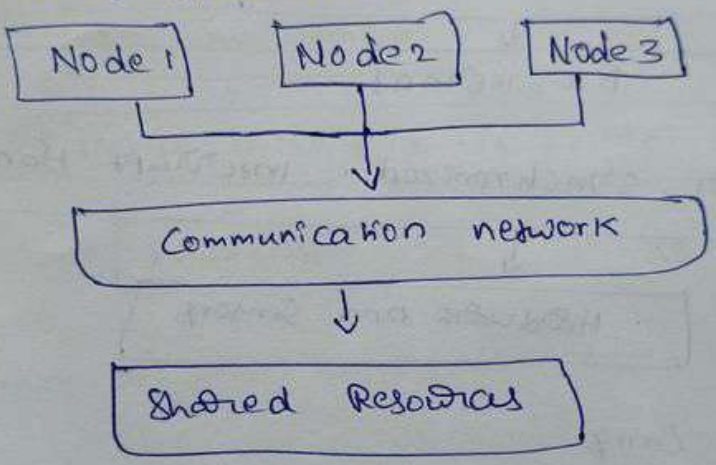

(c) System Calls

- Sem_wait() – waits for a semaphore
- Sem_post() – signals the semaphore
- Pthread_mutex_lock() – locks the mutex
- Pthread_mutex_unlock() – unlock the mutex
- Pthread_cond_wait() – wait for condition to become true

4) (a) Classification and comparison

| classification | Comparison |
| --- | --- |
| Distributed OS | Centralized OS |
| uses multiple connected computers | managed by one central computer |
| Resources are distributed across nodes. | Resources are managed centrally. |
| Can offer better scalability. | Expansion may depend. |

(b) Distributed OS Architechture



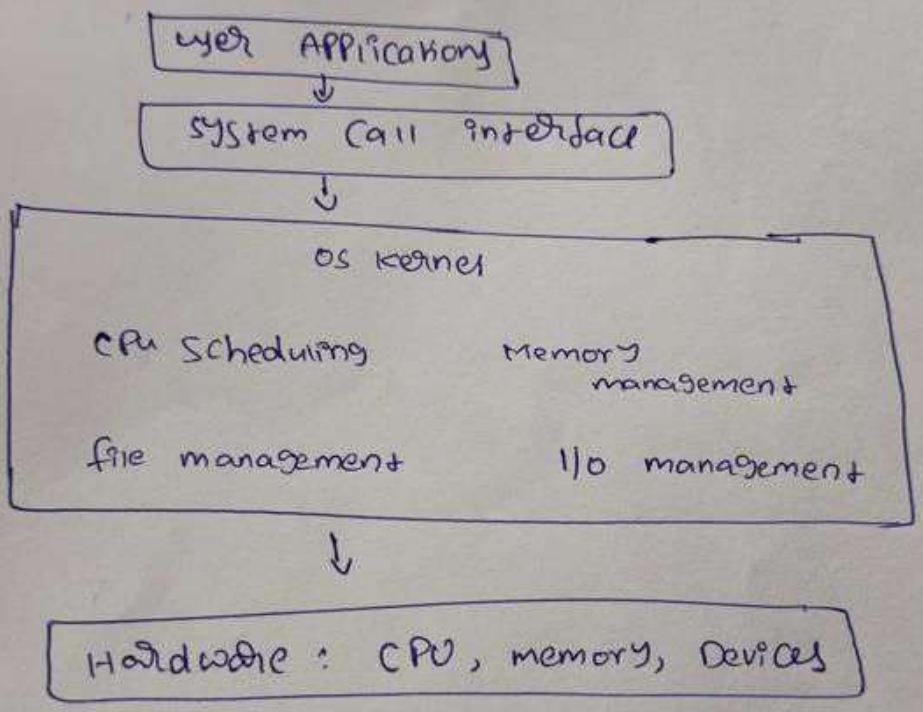

(c) System Calls for IPC

1) socket()
2) bind()
3) listen()
4) accept()
5) connect()
6) send and recv()
7) close()

(a) Major functions.

1) Process management
2) Memory management
3) Device management
4) File management
5) Security & Protection
6) Resource allocation.

(b) OS components block diagram




(c) System Call examples:

memory management

- mmap()
- munmap()
- brk()

Device management

- open()
- read()
- write()
- ioctl()
- close()